Module 4 : Modélisation UML

Diagrammes · Architecture · Synthèse

Python & Java · CompteBancaire M1+M2+M3

Diag. Classes | Diag. Séquence | Draw.io | PlantUML

MODULE 4 · Ingénierie Logicielle Évaluation : 40% TP · 60% Contrôle Continu

La Roadmap du Cursus : Ingénierie Logicielle

01
Modélisation Objet Avancée

Classes, Héritage, Encapsulation

02
Principe SOLID & Qualité

5 Principes SOLID, Refactoring

03
Design Patterns

Strategy, Observer, Singleton

04
Modélisation UML

Classes · Séquence
PlantUML · Draw.io

Objectif final : Concevoir · Implémenter · Tester · Livrer un logiciel de qualité industrielle

UML : Unified Modeling Language — le langage visuel des architectes logiciels

Définition

UML est un langage de modélisation graphique standardisé (ISO/IEC 19501). Représenter visuellement l'architecture d'un logiciel, indépendamment de Python ou de Java.

Pas du code : un SCHEMA.
UML offre 13 diagramme normalisé différents
Utilisé pour l'amelioration de processus

C'est le "Langage Universel" (Indépendant du code) :
L'UML est un **dessin standardisé** (ISO). Que tu travailles avec une équipe qui code en **Java** ou en **Python**, tout le monde comprend le schéma.

Il permet de valider l'architecture avec tes collègues avant même d'avoir choisi le langage de programmation.

Quand on code, on a la tête dans les détails (les points-virgules, les boucles). En commençant par l'UML : tu vois les classes et leurs relations , les interactions entre les objets , ça permet de repérer les oublier majeurs (relation client à son compte)
Permet d'avoir une architecture et un design propre (SOLID et Patterns)

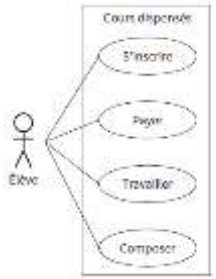
Objectif : lire ET écrire un diagramme sans maîtriser la norme complète.

Structure Diag. Classes Composants Déploiement	Comment le code est organisé et interconnecté entre classes.
Comportement Diag. Séquence Cas d'usage Activité	Comment les objets interagissent dans le temps (chronologie).
Ce module Diag. Classes ✓ Diag. Séquence ✓ PlantUML + Draw.io	Sur CompteBancaire M1+M2+M3 Synthèse des 3 modules.

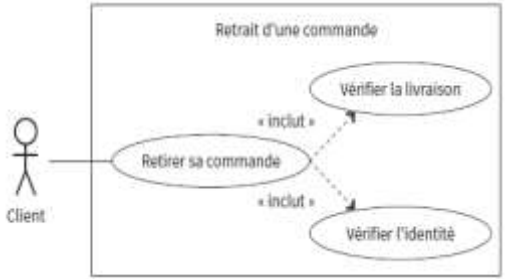
Fil rouge : l'UML de ce module représente le code écrit en M1 (POO), refactoré en M2 (SOLID) et enrichi en M3 (Patterns).

Exemple de diagrammes UML : Classes et Séquences

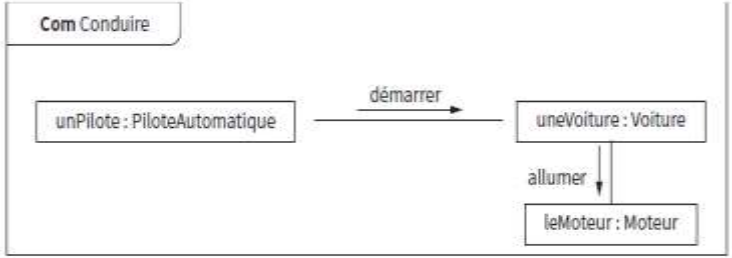
1- Diagramme minimaliste cas d'utilisation



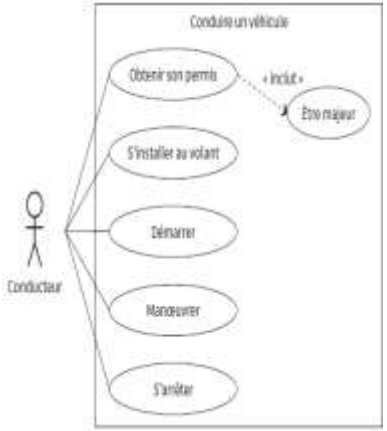
3 - Diagramme cas d'utilisation : relations d'exclusion



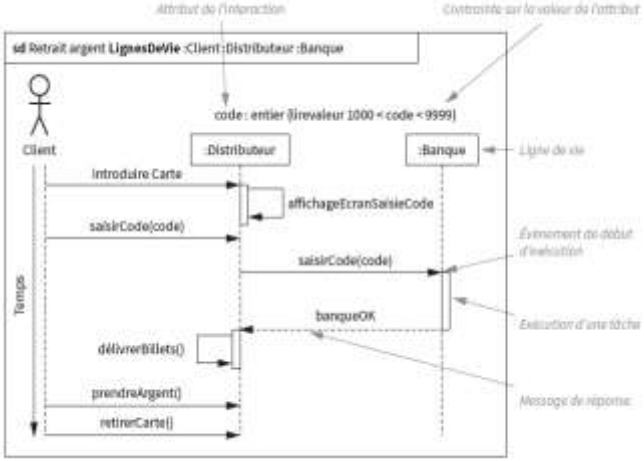
5 - Diagramme de communication : conduite automatique



2- Diagramme cas d'utilisation : relations d'inclusion



4- Diagramme de séquence d'un retrait d'argent à un distributeur de billets



6 - Diagramme d'activité : flots de contrôle magasin

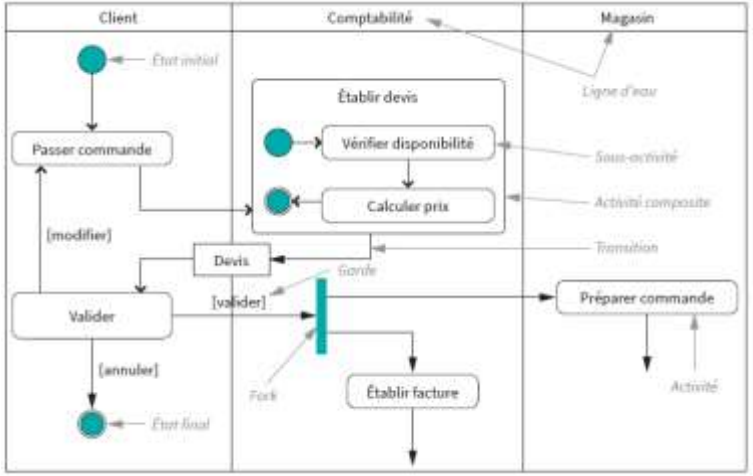


Diagramme de Classes

Structure statique du système

"Représenter toutes les classes, attributs, méthodes et relations"

Lire — Structure

Structure

Identifier classes, visibilité des membres (-/+) et relations : héritage, interface, association, composition.

Écrire — Modéliser

Modéliser

Créer le diagramme de CompteBancaire M1+M2+M3 : 8 classes + 3 interfaces + toutes les relations.

Outils — PlantUML

PlantUML

PlantUML : code texte vers diagramme.
Draw.io : glisser-déposer.
Les deux produisent UML standard.

Notation UML · Symboles Essentiels à Connaître

Définition diagramme de classe : C'est la photo d'identité de ton logiciel qui montre toutes les briques fixes (les classes) , ce qu'elles contiennent et comment elles sont reliées entre elles de façon permanente.

Notation UML : lire un diagramme de classes en 5 minutes

Visibilité des membres

+ public (accessible partout)
- private (inaccessible hors classe)
protected (accessible aux enfants)
Python : __attr = private (convention)

Relations entre classes

——————▸ Association (utilise-un)
——————▷ Héritage (est-un)
- - - - ▷ Réalisation d'interface
◆———— Composition (contient)

Multiplicités

1 exactement un
0..1 zéro ou un
0..* zéro ou plusieurs
1..* un ou plusieurs obligatoire

Anatomie d'une boîte UML — 3 compartiments obligatoires :

CompteBancaire

- __titulaire : str
- __solde : float
- __strategie : IFrais

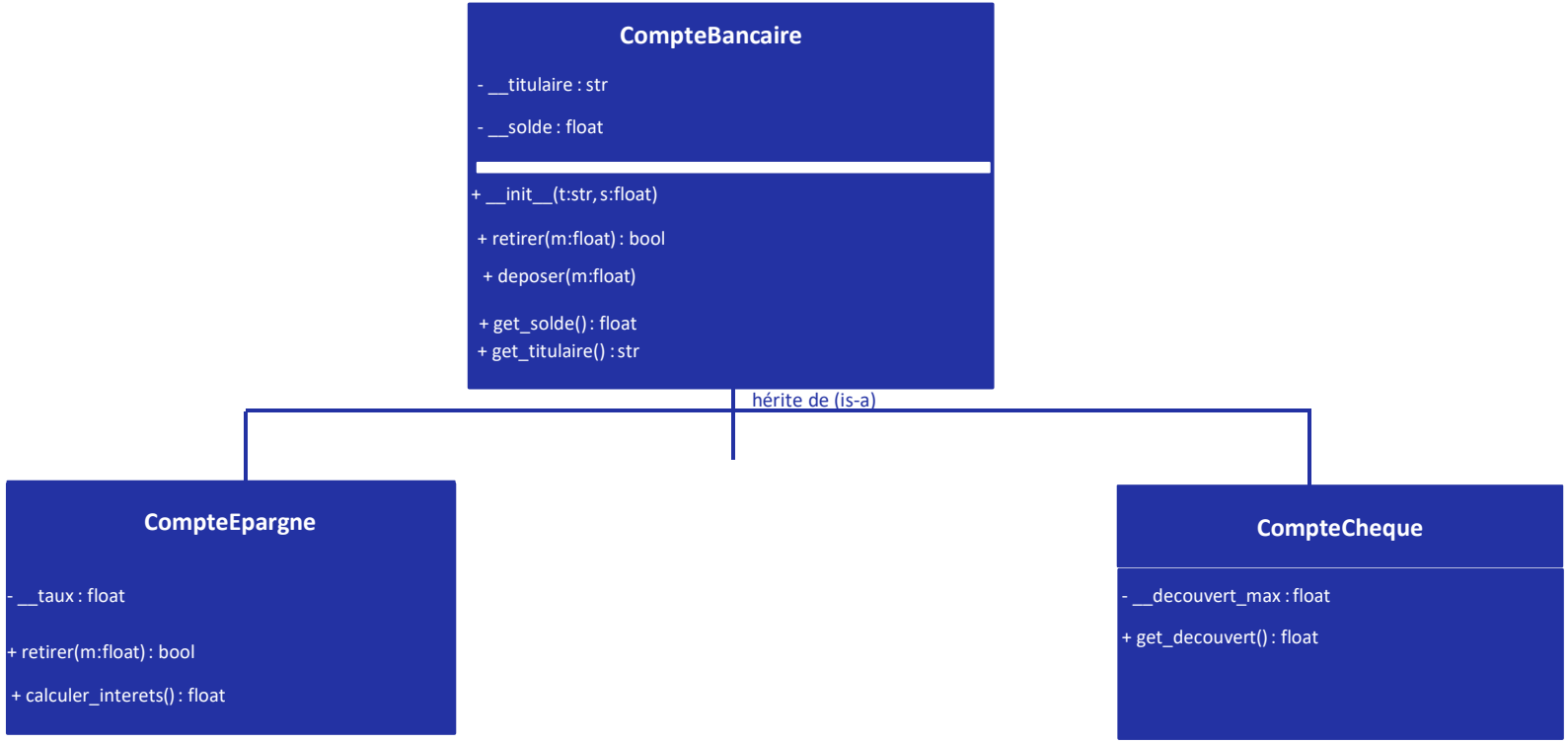
+ retirer(m:float) : bool
+ calculer_frais() : float
+ abonner(obs:IObserver)

Compartiment 1 : Nom de la classe (stéréotype si interface : «interface»)

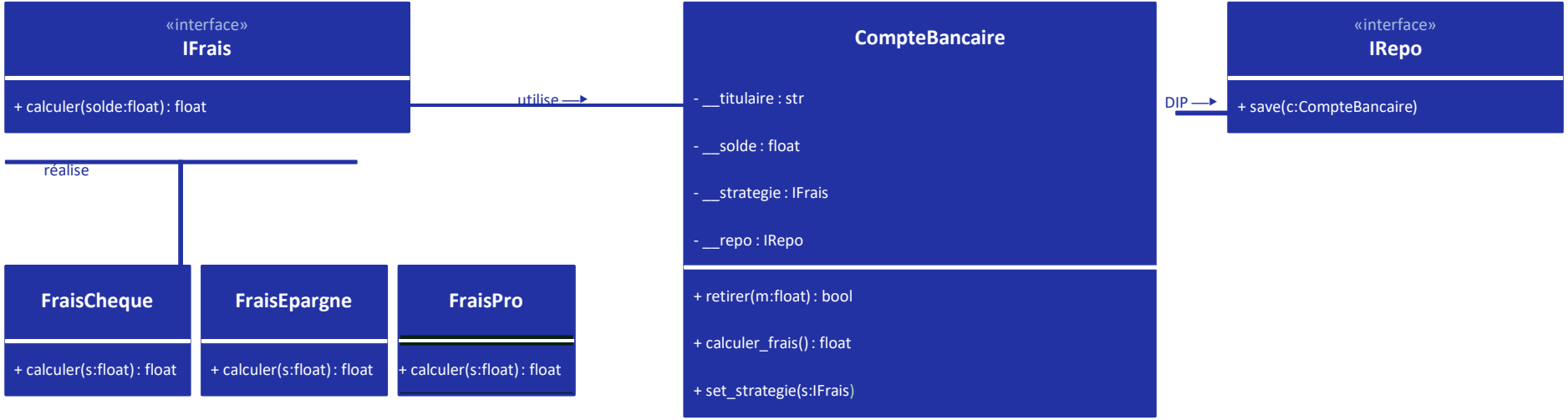
Compartiment 2 : Attributs visibilité nom : type
- = private + = public # = protected Python : __attr

Compartiment 3 : Méthodes visibilité nom(params) : type_retour
Convention : paramètres typés, type de retour obligatoire pour les fonctions.

M1 — Structure POO de base : CompteBancaire et ses sous-classes :

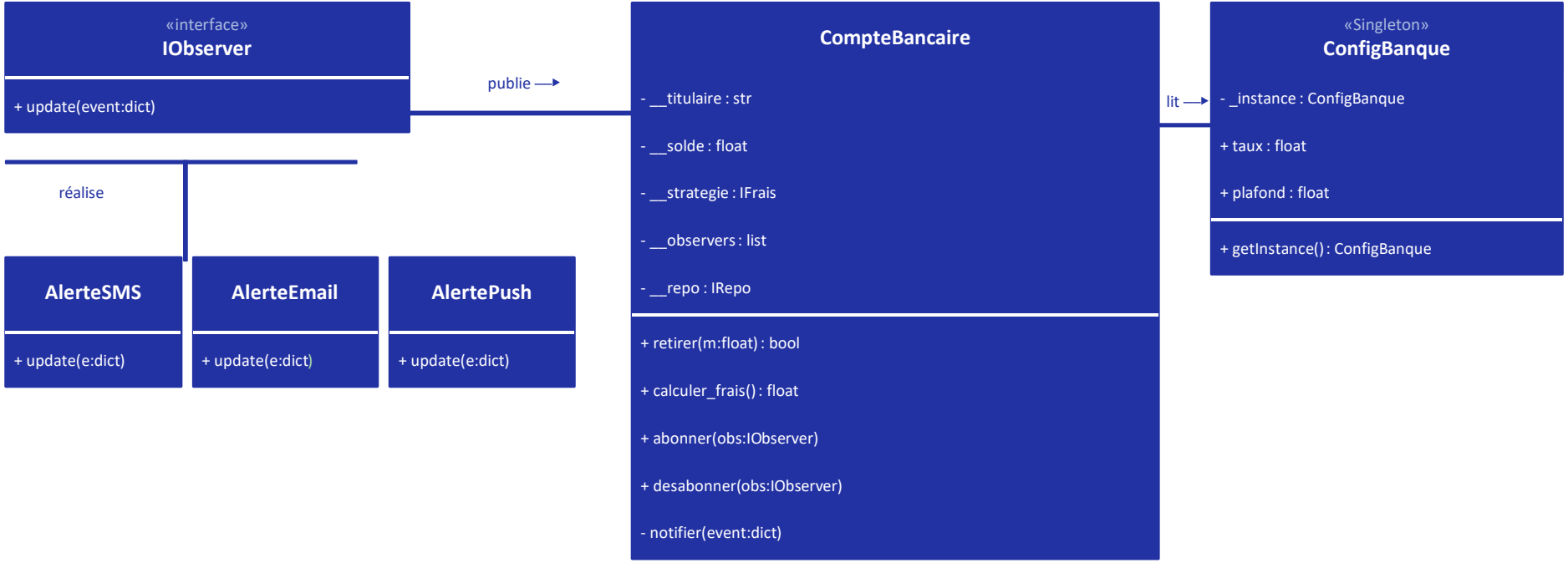


M2 — Ajout SOLID : IFrais (OCP) + IRepo (DIP) injectés dans CompteBancaire



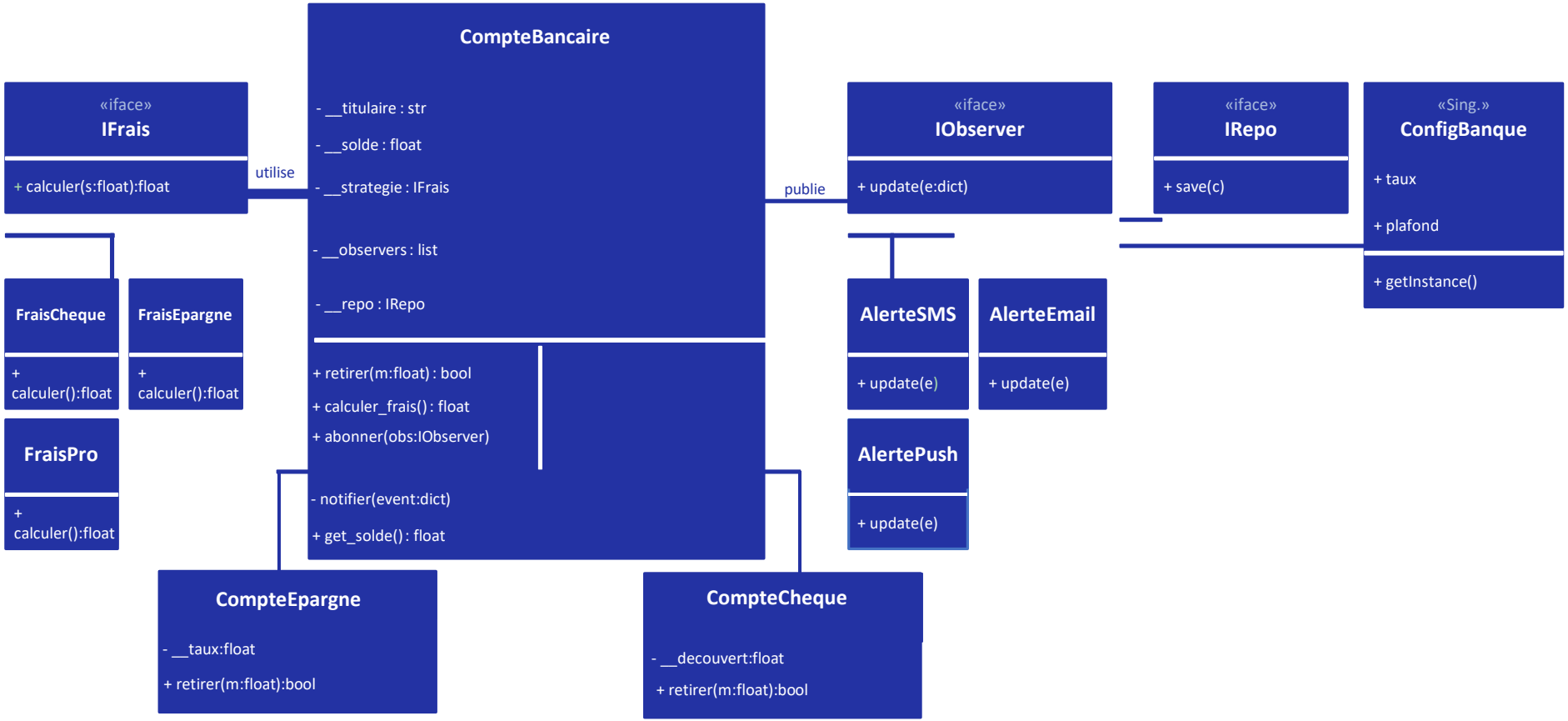
- - ▷ Réalisation : FraisCheque implémente IFrais. ⟶ Association : CompteBancaire utilise IFrais et IRepo (Dependency Inversion M2).

M3 — Ajout Patterns : IObserver (Observer) + ConfigBanque Singleton enrichissent CompteBancaire



Architecture finale M3 : CompteBancaire = Publisher (Observer) + Context (Strategy) + consomme Singleton ConfigBanque.

Vue synthétique : toutes les classes et interfaces du système bancaire final



▷ Héritage - -▷ Réalisation interface —▸ Association Vert=Strategy Bleu=Observer Marron=Singleton+DIP

Diagramme de Séquence

Flux dynamique du système

"Modéliser les échanges de messages entre objets dans le temps"

Quand — **Chronologie**

Chronologie

Le code seul ne montre pas l'ordre des appels lors de retirer(). Le diagramme de séquence visualise ce flux.

Structure — **Acteurs + Lifelines**

Acteurs + Lifelines

Acteurs horizontaux : Client, CompteBancaire, IFrais, IObserver... Temps = vertical. Messages = flèches.

Résultat — **Vue temporelle**

Vue temporelle

Voir exactement qui appelle qui, dans quel ordre, lors de compte.retirer(500) dans le système M1+M2+M3.

Définition Diagramme de Séquence : C’est le film de l’action qui montre , seconde après seconde , comment les objets s’envoient des messages et s’appellent les uns les uns les autres pour réaliser une tache précise comme un retrait d’argent .

Diagramme de séquence : 6 éléments à connaître pour le lire et l'écrire

1. Acteur (participant)

Chaque classe ou acteur = une colonne.
Représenté par un rectangle en haut.
Nommé : nomVariable:NomClasse

2. Ligne de vie (lifeline)

Trait vertical sous chaque acteur.
Représente l'existence dans le temps.

3. Message synchrone (flèche pleine)

——▸ appel de méthode.
- - - -▸ valeur de retour.

4. Boîte d'activation

Rectangle étroit sur la lifeline pendant qu'un objet traite un message.

5. Fragments combinés

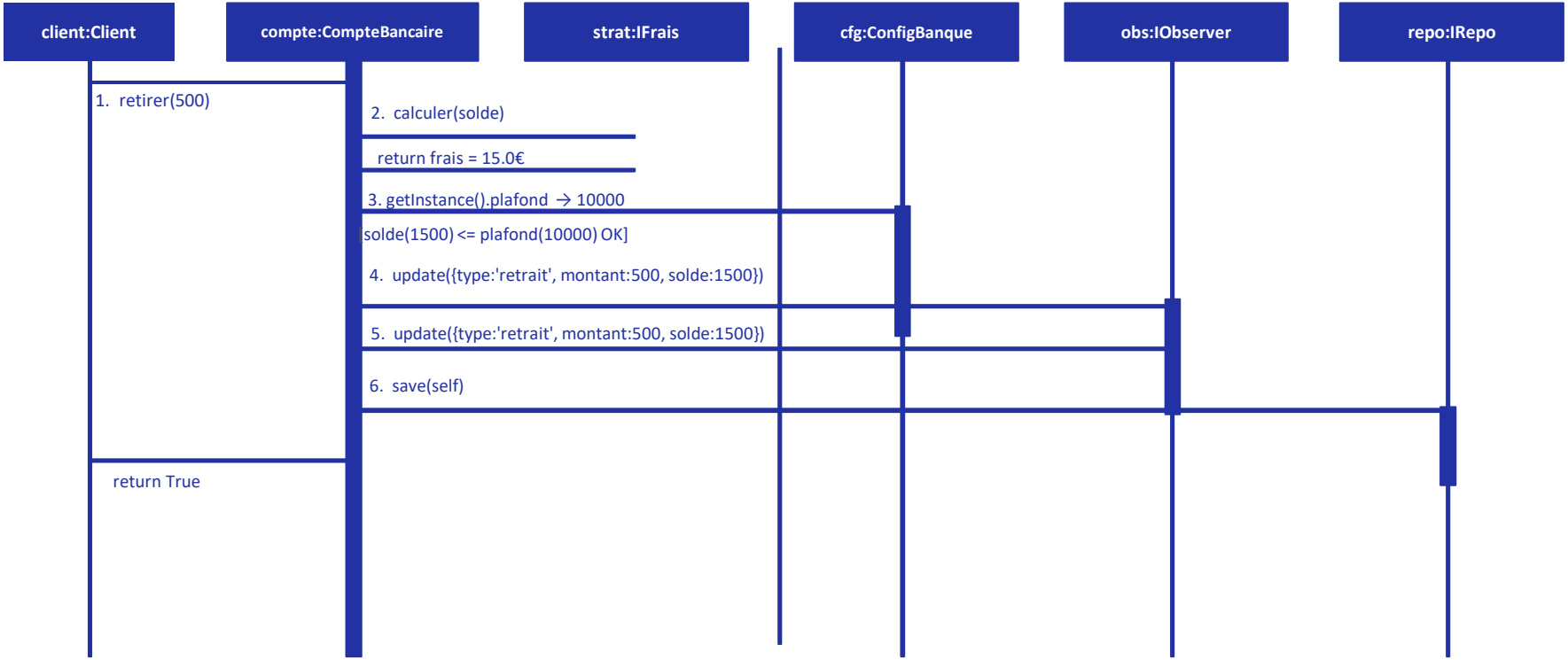
opt [condition] : exécuté si vrai.
loop [n] : répété n fois.
alt / else : comme if/else.

6. Ordre de lecture

De haut en bas = ordre chronologique.
De gauche à droite = acteurs impliqués.

Convention : acteurs nommés avec leur type (ex: client:Client, compte:CompteBancaire, obs1:IObserver).

Flux complet : que se passe-t-il lors de compte.retirer(500) dans le système final ?



Flux : Client → retirer() → calcul frais (Strategy) → lecture plafond (Singleton) → notifier Observers (×N) → sauvegarder (DIP) → True

Outils UML

PlantUML & Draw.io

"Coder ou glisser-déposer : deux chemins vers le même diagramme UML standard"

PlantUML — Code → Diagramme

Code → Diagramme

Syntaxe simple : @startuml/@enduml. Versionnable Git. VS Code + IntelliJ plugin. Génération automatique.

Draw.io — Glisser-Déposer

Glisser-Déposer

Interface graphique app.diagrams.net. Formes UML prédéfinies. Export PNG/SVG/PDF. Collaboration.

Choisir ? — Selon contexte

Selon contexte

Terminal Linux/Mac : PlantUML. Windows débutant : Draw.io. Les deux produisent UML standard valide.

PlantUML : syntaxe complète du diagramme de classes du système bancaire M1+M2+M3

Générer : plantuml fichier.puml (Mac : brew install plantuml · Linux : apt install plantuml · Windows : jar standalone)

PlantUML · Diagramme de Classes complet

BON

```
@startuml CompteBancaire_Classes

interface IFrais {
  + calculer(solde: float): float
}
interface IObserver {
  + update(event: dict)
}
interface IRepo {
  + save(c: CompteBancaire)
}

class CompteBancaire {
  - __titulaire: str
  - __solde: float
  - __strategie: IFrais
  - __observers: list
  + retirer(m: float): bool
  + calculer_frais(): float
  + abonner(obs: IObserver)
}

class FraisCheque implements IFrais
class FraisEpargne implements IFrais
class FraisPro implements IFrais
class AlerteSMS implements IObserver
class AlerteEmail implements IObserver

CompteBancaire --> IFrais : utilise
CompteBancaire --> IObserver : publie
CompteBancaire --> IRepo : sauvegarde
CompteBancaire <|-- CompteEpargne
CompteBancaire <|-- CompteCheque

@enduml
```

Comment utiliser PlantUML

1. Écrire le code dans fichier.puml
2. Générer l'image :
   `plantuml fichier.puml`

3. VS Code : extension PlantUML
   (prévisualisation live avec Alt+D)

4. IntelliJ : plugin PlantUML
   intégration native dans l'IDE

Résultat : PNG ou SVG du diagramme UML standard.

PlantUML : syntaxe du diagramme de séquence retirer(500) avec tous les participants

Syntaxe séquence : actor / participant / -> (message) / --> (retour) / activate / deactivate / note over.

PlantUML · Diagramme de Séquence retirer() BON

```
@startuml retirer_sequence

actor Client
participant "compte:CB" as CB
participant "strat:IFrais" as Strat
participant "cfg:ConfigBanque" as Cfg
participant "obs:IObserver" as Obs
participant "repo:IRepo" as Repo

Client -> CB: retirer(500)
activate CB

CB -> Strat: calculer(solde)
activate Strat
Strat --> CB: return 15.0
deactivate Strat

CB -> Cfg: getInstance().plafond
Cfg --> CB: 10000

note over CB: solde -= 500

CB -> Obs: update({montant:500})
CB -> Obs: update({montant:500})

CB -> Repo: save(self)
```

Syntaxe clé PlantUML Séquence

actor Nom → acteur externe
participant X as Y → renommer

```
A -> B: msg()    → appel
A --> B: val     → retour
```

activate A / deactivate A
→ boîte d'activation

opt [cond] / end → optionnel
loop [n] / end → répétition
alt / else / end → if/else

note over CB : texte interne

Générer : plantuml fichier.puml

Draw.io : créer le diagramme de classes CompteBancaire en glisser-déposer

Accès : app.diagrams.net (navigateur, sans installation) · Desktop : github.com/jgraph/drawio-desktop

1

Ouvrir Draw.io · Nouveau diagramme · Choisir template UML
Extras > Edit Diagram pour importer un XML existant, ou commencer avec les formes UML prédéfinies dans le panneau gauche.

2

Glisser une forme UML Class depuis le panneau gauche pour chaque classe
Double-clic sur le nom pour renommer. Ajouter attributs et méthodes dans les 2 compartiments inférieurs.

3

Tracer les relations : Inheritance (triangle vide) · Realization (pointillé+triangle) · Association (flèche)
Survoler le bord d'une classe → connecteur apparaît. Cliquer-glisser vers la classe cible pour tracer la relation.

4

Exporter : File > Export as > PNG/SVG/PDF · Partager le fichier .drawio (XML) pour collaboration
`Astuce Linux/Mac : draw.io --export --format png --output diagram.png fichier.drawio`

Raccourci : Ctrl+Shift+X → Éditeur XML direct. Coller votre code PlantUML converti. Plus rapide que glisser-déposer pour les grands diagrammes.

De l'UML au code : même diagramme, deux langages — correspondances directes

Notation UML	Python	Java
- attr : type	self.__attr = val (private)	private Type attr;
+ methode():T	def methode(self) -> T:	public T methode() { }
▷ Héritage	class A(B):	class A extends B {
- -▷ Réalisation	class A(IFrais):	class A implements IFrais {
—▶ Association	self.__obj = obj (attribut)	private B obj; (champ)
«interface»	class I: / Protocol	interface I {
«Singleton»	cls._instance (attribut de classe)	private static inst;

Synthèse · UML = Photo Finale des 4 Modules

Ce que montre le diagramme de classes final : les 4 modules en un seul schéma

Module 1 — POO socle

CompteBancaire + CompteEpargne + CompteCheque
Héritage (▷) + Encapsulation (attributs - privés)

Module 2 — Interfaces SOLID

IFrais (OCP) + IRepo (DIP) + ISP (interfaces séparées)
Réalisation (- -▷) + Association (—►)

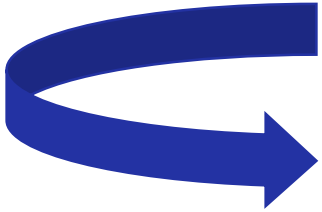
Module 3 — Design Patterns

IObserver (Observer) + ConfigBanque «Singleton»
Strategy : IFrais + 3 implémentations concrètes

Module 4 — Modélisation UML

Diagramme de classes : structure statique complète
Diagramme de séquence : flux retirer(500)

Lire le diagramme final =
comprendre les 4 modules de ce cours d'un seul coup d'œil.



Resumé pour l'ingenieur : Processus industrialisation logicielle

1- Modelisation (UML) : Le Plan de l'Architecte
2- Le mindset (SOLID) : Les Fondations de Qualités
3- Les Design Patterns : Les Solutions standardisées
4- La POO & Le TDD : La Construction Sécurisée

UML : Tableau Récapitulatif — Notation, Signification, Code Python et Java

Notation UML	Signification	Python	Java
- attr : type	Attribut privé	self.__attr = val	private Type attr;
+ methode():T	Méthode publique	def methode(self) -> T:	public T methode() { }
▷ Héritage	Est-un (is-a)	class A(B):	class A extends B {
- -▷ Réalisation	Implémente interface	class A(IFrais):	class A implements IFrais {
—▶ Association	Utilise-un (attribut)	self.__obj = obj	private B obj;
«interface»	Contrat abstrait	class I: / ABC	interface I {
«Singleton»	Instance unique globale	cls._instance	private static inst;

TP — Modéliser le système Gestion Etudiants complet du TP1 avec PlantUML ou Draw.io en 2h

Livrable : 1 diagramme de classes complet (M1+M2+M3) + 1 diagramme de séquence Gestion Etudiants Export PDF ou PNG.

1 **Diagramme de Classes · Toutes les classes M1+M2+M3**

2 **Diagramme de Classes · Toutes les relations**

3 **Diagramme de Séquence · retirer(500) flux complet**

4 **Export et rendu**
PlantUML : plantuml fichier.puml → PNG. Draw.io : File > Export > PDF. Nommer : Groupe_UML_M4.pdf.

Durée estimée : Diag. Classes 60min · Diag. Séquence 45min · Export + vérification 15min

Ressources UML : outils, documentation et références pour aller plus loin

PlantUML

plantuml.com

Référence officielle
Plugin VS Code
Plugin IntelliJ
Mode live preview

apt install plantuml
brew install plantuml

Draw.io

app.diagrams.net

Interface web gratuite
Desktop : GitHub
Export PNG/SVG/PDF
Import XML/PlantUML

Ctrl+Shift+X = édit XML

UML Spec

omg.org/spec/UML

Norme officielle OMG
14 types de diagrammes
Pas nécessaire en entier
Classes + Séquence = 80%

Livre conseillé : UML2

Cours suivants

koor.fr

3ème année :
Diag. cas d'usage
Diag. état-transition
Patterns avancés

Tests unitaires TDD

Ce module complète le cursus 2A. Vous pouvez maintenant lire, écrire et discuter l'architecture logicielle avec les outils UML standard.

Fin du Module 4

Place à la Pratique UML !

Travaux Pratiques (TP) : PlantUML & Draw.io

Objectif : Diagramme de classes M1+M2+M3 + Diagramme de séquence TP1 Gestion Etudiants · 2h

01

Diag. Classes
M1+M2+M3

02

Diag. Séquence
retirer(500)

03

PlantUML
ou
Draw.io

04

Export PDF/PNG
Rendu final

koor.fr
plantuml.com
diagrams.net